
textblob Documentation

Release 0.20.0

Steven Loria

May 21, 2026

CONTENTS

1	Features	3
2	Get it now	5
3	Guide	7
3.1	License	7
3.2	Installation	7
3.3	Tutorial: Quickstart	9
3.4	Tutorial: Building a Text Classification System	13
3.5	Advanced Usage: Overriding Models and the Blobber Class	17
3.6	Extensions	19
3.7	API Reference	19
4	Project info	55
	Python Module Index	57
	Index	59

Release v0.20.0. (Changelog)

TextBlob is a Python library for processing textual data. It provides a simple API for diving into common natural language processing (NLP) tasks such as part-of-speech tagging, noun phrase extraction, sentiment analysis, classification, and more.

```
from textblob import TextBlob

text = """
The titular threat of The Blob has always struck me as the ultimate movie
monster: an insatiably hungry, amoeba-like mass able to penetrate
virtually any safeguard, capable of--as a doomed doctor chillingly
describes it--"assimilating flesh on contact.
Snide comparisons to gelatin be damned, it's a concept with the most
devastating of potential consequences, not unlike the grey goo scenario
proposed by technological theorists fearful of
artificial intelligence run rampant.
"""

blob = TextBlob(text)
blob.tags # [('The', 'DT'), ('titular', 'JJ'),
# ('threat', 'NN'), ('of', 'IN'), ...]

blob.noun_phrases # WordList(['titular threat', 'blob',
# 'ultimate movie monster',
# 'amoeba-like mass', ...])

for sentence in blob.sentences:
    print(sentence.sentiment.polarity)
# 0.060
# -0.341
```

TextBlob stands on the giant shoulders of [NLTK](#) and [pattern](#), and plays nicely with both.

FEATURES

- Noun phrase extraction
- Part-of-speech tagging
- Sentiment analysis
- Classification (Naive Bayes, Decision Tree)
- Tokenization (splitting text into words and sentences)
- Word and phrase frequencies
- Parsing
- n-grams
- Word inflection (pluralization and singularization) and lemmatization
- Spelling correction
- Add new models or languages through extensions
- WordNet integration

GET IT NOW

```
$ pip install -U textblob  
$ python -m textblob.download_corpora
```

Ready to dive in? Go on to the *[Quickstart guide](#)*.

3.1 License

Copyright Steven Loria **and** contributors

Permission **is** hereby granted, free of charge, to **any** person obtaining a copy of this software **and** associated documentation files (the "**Software**"), to deal **in** the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, **and/or** sell copies of the Software, **and** to permit persons to whom the Software **is** furnished to do so, subject to the following conditions:

The above copyright notice **and** this permission notice shall be included **in** all copies **or** substantial portions of the Software.

THE SOFTWARE IS PROVIDED "**AS IS**", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

3.2 Installation

3.2.1 Installing/Upgrading From the PyPI

```
$ pip install -U textblob
$ python -m textblob.download_corpora
```

This will install TextBlob and download the necessary NLTK corpora. If you need to change the default download directory set the NLTK_DATA environment variable.

Downloading the minimum corpora

If you only intend to use TextBlob's default models (no model overrides), you can pass the `lite` argument. This downloads only those corpora needed for basic functionality.

```
$ python -m textblob.download_corpora lite
```

3.2.2 With conda

TextBlob is also available as a [conda](#) package. To install with conda, run

```
$ conda install -c conda-forge textblob
$ python -m textblob.download_corpora
```

3.2.3 From Source

TextBlob is actively developed on [Github](#).

You can clone the public repo:

```
$ git clone https://github.com/sloria/TextBlob.git
```

Or download one of the following:

- [tarball](#)
- [zipball](#)

Once you have the source, you can install it into your site-packages with

```
$ python setup.py install
```

3.2.4 Get the bleeding edge version

To get the latest development version of TextBlob, run

```
$ pip install -U git+https://github.com/sloria/TextBlob.git@dev
```

3.2.5 Migrating from older versions (<=0.7.1)

As of TextBlob 0.8.0, TextBlob's core package was renamed to `textblob`, whereas earlier versions used a package called `text`. Therefore, migrating to newer versions should be as simple as rewriting your imports, like so:

New:

```
from textblob import TextBlob, Word, Blobber
from textblob.classifiers import NaiveBayesClassifier
from textblob.taggers import NLTKTagger
```

Old:

```
from text.blob import TextBlob, Word, Blobber
from text.classifiers import NaiveBayesClassifier
from text.taggers import NLTKTagger
```

Dependencies

TextBlob depends on NLTK 3. NLTK will be installed automatically when you run `pip install textblob`.

Some features, such as the maximum entropy classifier, require [numpy](#), but it is not required for basic usage.

3.3 Tutorial: Quickstart

TextBlob aims to provide access to common text-processing operations through a familiar interface. You can treat *TextBlob* objects as if they were Python strings that learned how to do Natural Language Processing.

3.3.1 Create a TextBlob

First, the import.

```
>>> from textblob import TextBlob
```

Let's create our first *TextBlob*.

```
>>> wiki = TextBlob("Python is a high-level, general-purpose programming language.")
```

3.3.2 Part-of-speech Tagging

Part-of-speech tags can be accessed through the *tags* property.

```
>>> wiki.tags
[('Python', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('high-level', 'JJ'), ('general-purpose',
→ 'JJ'), ('programming', 'NN'), ('language', 'NN')]
```

3.3.3 Noun Phrase Extraction

Similarly, noun phrases are accessed through the *noun_phrases* property.

```
>>> wiki.noun_phrases
WordList(['python'])
```

3.3.4 Sentiment Analysis

The *sentiment* property returns a namedtuple of the form `Sentiment(polarity, subjectivity)`. The polarity score is a float within the range [-1.0, 1.0]. The subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

```
>>> testimonial = TextBlob("Textblob is amazingly simple to use. What great fun!")
>>> testimonial.sentiment
Sentiment(polarity=0.39166666666666666, subjectivity=0.4357142857142857)
>>> testimonial.sentiment.polarity
0.39166666666666666
```

3.3.5 Tokenization

You can break TextBlobs into words or sentences.

```
>>> zen = TextBlob(
...     "Beautiful is better than ugly. "
...     "Explicit is better than implicit. "
...     "Simple is better than complex."
... )
>>> zen.words
WordList(['Beautiful', 'is', 'better', 'than', 'ugly', 'Explicit', 'is', 'better', 'than
```

(continues on next page)

(continued from previous page)

```
↪', 'implicit', 'Simple', 'is', 'better', 'than', 'complex'])
>>> zen.sentences
[Sentence("Beautiful is better than ugly."), Sentence("Explicit is better than implicit.
↪"), Sentence("Simple is better than complex.")]
```

`Sentence` objects have the same properties and methods as `TextBlobs`.

```
>>> for sentence in zen.sentences:
...     print(sentence.sentiment)
```

For more advanced tokenization, see the *Advanced Usage* guide.

3.3.6 Words Inflection and Lemmatization

Each word in `TextBlob.words` or `Sentence.words` is a `Word` object (a subclass of `unicode`) with useful methods, e.g. for word inflection.

```
>>> sentence = TextBlob("Use 4 spaces per indentation level.")
>>> sentence.words
WordList(['Use', '4', 'spaces', 'per', 'indentation', 'level'])
>>> sentence.words[2].singularize()
'space'
>>> sentence.words[-1].pluralize()
'levels'
```

Words can be lemmatized by calling the `lemmatize` method.

```
>>> from textblob import Word
>>> w = Word("octopi")
>>> w.lemmatize()
'octopus'
>>> w = Word("went")
>>> w.lemmatize("v") # Pass in WordNet part of speech (verb)
'go'
```

3.3.7 WordNet Integration

You can access the synsets for a `Word` via the `synsets` property or the `get_synsets` method, optionally passing in a part of speech.

```
>>> from textblob import Word
>>> from textblob.wordnet import VERB
>>> word = Word("octopus")
>>> word.synsets
[Synset('octopus.n.01'), Synset('octopus.n.02')]
>>> Word("hack").get_synsets(pos=VERB)
[Synset('chop.v.05'), Synset('hack.v.02'), Synset('hack.v.03'), Synset('hack.v.04'), ↵
↪Synset('hack.v.05'), Synset('hack.v.06'), Synset('hack.v.07'), Synset('hack.v.08')]
```

You can access the definitions for each synset via the `definitions` property or the `define()` method, which can also take an optional part-of-speech argument.

```
>>> Word("octopus").definitions
['tentacles of octopus prepared as food', 'bottom-living cephalopod having a soft oval_
↳ body with eight long tentacles']
```

You can also create synsets directly.

```
>>> from textblob.wordnet import Synset
>>> octopus = Synset("octopus.n.02")
>>> shrimp = Synset("shrimp.n.03")
>>> octopus.path_similarity(shrimp)
0.11111111111111111
```

For more information on the WordNet API, see the NLTK documentation on the [Wordnet Interface](#).

3.3.8 WordLists

A `WordList` is just a Python list with additional methods.

```
>>> animals = TextBlob("cat dog octopus")
>>> animals.words
WordList(['cat', 'dog', 'octopus'])
>>> animals.words.pluralize()
WordList(['cats', 'dogs', 'octopodes'])
```

3.3.9 Spelling Correction

Use the `correct()` method to attempt spelling correction.

```
>>> b = TextBlob("I havv goood speling!")
>>> print(b.correct())
I have good spelling!
```

`Word` objects have a `spellcheck()` `Word.spellcheck()` method that returns a list of (word, confidence) tuples with spelling suggestions.

```
>>> from textblob import Word
>>> w = Word("falibility")
>>> w.spellcheck()
[('fallibility', 1.0)]
```

Spelling correction is based on Peter Norvig’s “How to Write a Spelling Corrector”¹ as implemented in the pattern library. It is about 70% accurate².

3.3.10 Get Word and Noun Phrase Frequencies

There are two ways to get the frequency of a word or noun phrase in a `TextBlob`.

The first is through the `word_counts` dictionary.

```
>>> monty = TextBlob("We are no longer the Knights who say Ni. "
...                  "We are now the Knights who say Ekki ekki ekki PTANG.")
```

(continues on next page)

¹ <http://norvig.com/spell-correct.html>

² <http://www.clips.ua.ac.be/pages/pattern-en#spelling>

(continued from previous page)

```
>>> monty.word_counts['ekki']
3
```

If you access the frequencies this way, the search will *not* be case sensitive, and words that are not found will have a frequency of 0.

The second way is to use the `count()` method.

```
>>> monty.words.count('ekki')
3
```

You can specify whether or not the search should be case-sensitive (default is `False`).

```
>>> monty.words.count('ekki', case_sensitive=True)
2
```

Each of these methods can also be used with noun phrases.

```
>>> wiki.noun_phrases.count('python')
1
```

3.3.11 Parsing

Use the `parse()` method to parse the text.

```
>>> b = TextBlob("And now for something completely different.")
>>> print(b.parse())
And/CC/O/O now/RB/B-ADVP/O for/IN/B-PP/B-PNP something/NN/B-NP/I-PNP completely/RB/B-
↳ADJP/O different/JJ/I-ADJP/O ././O/O
```

By default, TextBlob uses pattern's parser³.

3.3.12 TextBlobs Are Like Python Strings!

You can use Python's substring syntax.

```
>>> zen[0:19]
TextBlob("Beautiful is better")
```

You can use common string methods.

```
>>> zen.upper()
TextBlob("BEAUTIFUL IS BETTER THAN UGLY. EXPLICIT IS BETTER THAN IMPLICIT. SIMPLE IS_
↳BETTER THAN COMPLEX.")
>>> zen.find("Simple")
65
```

You can make comparisons between TextBlobs and strings.

```
>>> apple_blob = TextBlob("apples")
>>> banana_blob = TextBlob("bananas")
>>> apple_blob < banana_blob
```

(continues on next page)

³ <http://www.clips.ua.ac.be/pages/pattern-en#parser>

(continued from previous page)

```
True
>>> apple_blob == "apples"
True
```

You can concatenate and interpolate TextBlobs and strings.

```
>>> apple_blob + " and " + banana_blob
TextBlob("apples and bananas")
>>> "{0} and {1}".format(apple_blob, banana_blob)
'apples and bananas'
```

3.3.13 n-grams

The `TextBlob.ngrams()` method returns a list of tuples of n successive words.

```
>>> blob = TextBlob("Now is better than never.")
>>> blob.ngrams(n=3)
[WordList(['Now', 'is', 'better']), WordList(['is', 'better', 'than']), WordList(['better', 'than', 'never'])]
```

3.3.14 Get Start and End Indices of Sentences

Use `sentence.start` and `sentence.end` to get the indices where a sentence starts and ends within a `TextBlob`.

```
>>> for s in zen.sentences:
...     print(s)
...     print("---- Starts at index {}, Ends at index {}".format(s.start, s.end))
...
Beautiful is better than ugly.
---- Starts at index 0, Ends at index 30
Explicit is better than implicit.
---- Starts at index 31, Ends at index 64
Simple is better than complex.
---- Starts at index 65, Ends at index 95
```

Next Steps

Want to build your own text classification system? Check out the [Classifiers Tutorial](#).

Want to use a different POS tagger or noun phrase chunker implementation? Check out the [Advanced Usage](#) guide.

3.4 Tutorial: Building a Text Classification System

The `textblob.classifiers` module makes it simple to create custom classifiers.

As an example, let's create a custom sentiment analyzer.

3.4.1 Loading Data and Creating a Classifier

First we'll create some training and test data.

```
>>> train = [
...     ("I love this sandwich.", "pos"),
...     ("this is an amazing place!", "pos"),
...     ("I feel very good about these beers.", "pos"),
...     ("this is my best work.", "pos"),
...     ("what an awesome view", "pos"),
...     ("I do not like this restaurant", "neg"),
...     ("I am tired of this stuff.", "neg"),
...     ("I can't deal with this", "neg"),
...     ("he is my sworn enemy!", "neg"),
...     ("my boss is horrible.", "neg"),
... ]
>>> test = [
...     ("the beer was good.", "pos"),
...     ("I do not enjoy my job", "neg"),
...     ("I ain't feeling dandy today.", "neg"),
...     ("I feel amazing!", "pos"),
...     ("Gary is a friend of mine.", "pos"),
...     ("I can't believe I'm doing this.", "neg"),
... ]
```

Now we'll create a Naive Bayes classifier, passing the training data into the constructor.

```
>>> from textblob.classifiers import NaiveBayesClassifier
>>> cl = NaiveBayesClassifier(train)
```

Loading Data from Files

You can also load data from common file formats including CSV, JSON, and TSV.

CSV files should be formatted like so:

```
I love this sandwich.,pos
This is an amazing place!,pos
I do not like this restaurant,neg
```

JSON files should be formatted like so:

```
[
  {"text": "I love this sandwich.", "label": "pos"},
  {"text": "This is an amazing place!", "label": "pos"},
  {"text": "I do not like this restaurant", "label": "neg"}
]
```

You can then pass the opened file into the constructor.

```
>>> with open('train.json', 'r') as fp:
...     cl = NaiveBayesClassifier(fp, format="json")
```

3.4.2 Classifying Text

Call the `classify(text)` method to use the classifier.

```
>>> cl.classify("This is an amazing library!")
'pos'
```

You can get the label probability distribution with the `prob_classify(text)` method.

```
>>> prob_dist = cl.prob_classify("This one's a doozy.")
>>> prob_dist.max()
'pos'
>>> round(prob_dist.prob("pos"), 2)
0.63
>>> round(prob_dist.prob("neg"), 2)
0.37
```

3.4.3 Classifying TextBlobs

Another way to classify text is to pass a classifier into the constructor of `TextBlob` and call its `classify()` method.

```
>>> from textblob import TextBlob
>>> blob = TextBlob("The beer is good. But the hangover is horrible.", classifier=cl)
>>> blob.classify()
'pos'
```

The advantage of this approach is that you can classify sentences within a `TextBlob`.

```
>>> for s in blob.sentences:
...     print(s)
...     print(s.classify())
...
The beer is good.
pos
But the hangover is horrible.
neg
```

3.4.4 Evaluating Classifiers

To compute the accuracy on our test set, use the `accuracy(test_data)` method.

```
>>> cl.accuracy(test)
0.8333333333333334
```

Note

You can also pass in a file object into the `accuracy` method. The file can be in any of the formats listed in the [Loading Data](#) section.

Use the `show_informative_features()` method to display a listing of the most informative features.

```
>>> cl.show_informative_features(5)
Most Informative Features
contains(my) = True          neg : pos    =      1.7 : 1.0
contains(an) = False        neg : pos    =      1.6 : 1.0
```

(continues on next page)

(continued from previous page)

contains(I) = True	neg : pos	=	1.4 : 1.0
contains(I) = False	pos : neg	=	1.4 : 1.0
contains(my) = False	pos : neg	=	1.3 : 1.0

3.4.5 Updating Classifiers with New Data

Use the `update(new_data)` method to update a classifier with new training data.

```
>>> new_data = [
...     ("She is my best friend.", "pos"),
...     ("I'm happy to have a new friend.", "pos"),
...     ("Stay thirsty, my friend.", "pos"),
...     ("He ain't from around here.", "neg"),
... ]
>>> cl.update(new_data)
True
>>> cl.accuracy(test)
1.0
```

3.4.6 Feature Extractors

By default, the `NaiveBayesClassifier` uses a simple feature extractor that indicates which words in the training set are contained in a document.

For example, the sentence “*I feel happy*” might have the features `contains(happy): True` or `contains(angry): False`.

You can override this feature extractor by writing your own. A feature extractor is simply a function with `document` (the text to extract features from) as the first argument. The function may include a second argument, `train_set` (the training dataset), if necessary.

The function should return a dictionary of features for document.

For example, let’s create a feature extractor that just uses the first and last words of a document as its features.

```
>>> def end_word_extractor(document):
...     tokens = document.split()
...     first_word, last_word = tokens[0], tokens[-1]
...     feats = {}
...     feats["first({})".format(first_word)] = True
...     feats["last({})".format(last_word)] = False
...     return feats
...
>>> features = end_word_extractor("I feel happy")
>>> assert features == {"last(happy)": False, "first(I)": True}
```

We can then use the feature extractor in a classifier by passing it as the second argument of the constructor.

```
>>> cl2 = NaiveBayesClassifier(test, feature_extractor=end_word_extractor)
>>> blob = TextBlob("I'm excited to try my new classifier.", classifier=cl2)
>>> blob.classify()
'pos'
```

3.4.7 Next Steps

Be sure to check out the [API Reference](#) for the *classifiers module*.

Want to try different POS taggers or noun phrase chunkers with TextBlobs? Check out the [Advanced Usage](#) guide.

3.5 Advanced Usage: Overriding Models and the Blobber Class

TextBlob allows you to specify which algorithms you want to use under the hood of its simple API.

3.5.1 Sentiment Analyzers

New in version 0.5.0.

The `textblob.sentiments` module contains two sentiment analysis implementations, `PatternAnalyzer` (based on the `pattern` library) and `NaiveBayesAnalyzer` (an `NLTK` classifier trained on a movie reviews corpus).

The default implementation is `PatternAnalyzer`, but you can override the analyzer by passing another implementation into a `TextBlob`'s constructor.

For instance, the `NaiveBayesAnalyzer` returns its result as a `namedtuple` of the form: `Sentiment(classification, p_pos, p_neg)`.

```
>>> from textblob import TextBlob
>>> from textblob.sentiments import NaiveBayesAnalyzer
>>> blob = TextBlob("I love this library", analyzer=NaiveBayesAnalyzer())
>>> blob.sentiment
Sentiment(classification='pos', p_pos=0.7996209910191279, p_neg=0.2003790089808724)
```

3.5.2 Tokenizers

New in version 0.4.0.

The `words` and `sentences` properties are helpers that use the `textblob.tokenizers.WordTokenizer` and `textblob.tokenizers.SentenceTokenizer` classes, respectively.

You can use other tokenizers, such as those provided by `NLTK`, by passing them into the `TextBlob` constructor then accessing the `tokens` property.

```
>>> from textblob import TextBlob
>>> from nltk.tokenize import TabTokenizer
>>> tokenizer = TabTokenizer()
>>> blob = TextBlob("This is\ta rather tabby\tblob.", tokenizer=tokenizer)
>>> blob.tokens
WordList(['This is', 'a rather tabby', 'blob.'])
```

You can also use the `tokenize([tokenizer])` method.

```
>>> from textblob import TextBlob
>>> from nltk.tokenize import BlanklineTokenizer
>>> tokenizer = BlanklineTokenizer()
>>> blob = TextBlob("A token\n\nof appreciation")
>>> blob.tokenize(tokenizer)
WordList(['A token', 'of appreciation'])
```

3.5.3 Noun Phrase Chunkers

TextBlob currently has two noun phrases chunker implementations, `textblob.np_extractors.FastNPExtractor` (default, based on Shlomi Babluki's implementation from [this blog post](#)) and `textblob.np_extractors.ConllExtractor`, which uses the CoNLL 2000 corpus to train a tagger.

You can change the chunker implementation (or even use your own) by explicitly passing an instance of a noun phrase extractor to a TextBlob's constructor.

```
>>> from textblob import TextBlob
>>> from textblob.np_extractors import ConllExtractor
>>> extractor = ConllExtractor()
>>> blob = TextBlob("Python is a high-level programming language.", np_
↳ extractor=extractor)
>>> blob.noun_phrases
WordList(['python', 'high-level programming language'])
```

3.5.4 POS Taggers

TextBlob currently has two POS tagger implementations, located in `textblob.taggers`. The default is the `PatternTagger` which uses the same implementation as the `pattern` library.

The second implementation is `NLTKTagger` which uses NLTK's TreeBank tagger. *Numpy is required to use the NLTK-Tagger.*

Similar to the tokenizers and noun phrase chunkers, you can explicitly specify which POS tagger to use by passing a tagger instance to the constructor.

```
>>> from textblob import TextBlob
>>> from textblob.taggers import NLTKTagger
>>> nltk_tagger = NLTKTagger()
>>> blob = TextBlob("Tag! You're It!", pos_tagger=nltk_tagger)
>>> blob.pos_tags
[(Word('Tag'), u'NN'), (Word('You'), u'PRP'), (Word(''), u'VBZ'), (Word('re'), u'NN'),
↳ (Word('It')
, u'PRP')]
```

3.5.5 Parsers

New in version 0.6.0.

Parser implementations can also be passed to the TextBlob constructor.

```
>>> from textblob import TextBlob
>>> from textblob.parsers import PatternParser
>>> blob = TextBlob("Parsing is fun.", parser=PatternParser())
>>> blob.parse()
'Parsing/VBG/B-VP/O is/VBZ/I-VP/O fun/VBG/I-VP/O ././O/O'
```

3.5.6 Blobber: A TextBlob Factory

New in 0.4.0.

It can be tedious to repeatedly pass taggers, NP extractors, sentiment analyzers, classifiers, and tokenizers to multiple TextBlobs. To keep your code [DRY](#), you can use the `Blobber` class to create TextBlobs that share the same models.

First, instantiate a `Blobber` with the tagger, NP extractor, sentiment analyzer, classifier, and/or tokenizer of your choice.

```
>>> from textblob import Blobber
>>> from textblob.taggers import NLTKTagger
>>> tb = Blobber(pos_tagger=NLTKTagger())
```

You can now create new TextBlobs like so:

```
>>> blob1 = tb("This is a blob.")
>>> blob2 = tb("This is another blob.")
>>> blob1.pos_tagger is blob2.pos_tagger
True
```

3.6 Extensions

TextBlob supports adding custom models and new languages through “extensions”.

Extensions can be installed from the PyPI.

```
$ pip install textblob-name
```

where “name” is the name of the package.

3.6.1 Available extensions

Languages

- `textblob-fr`: French
- `textblob-de`: German

Part-of-speech Taggers

- `textblob-aptagger`: A fast and accurate tagger based on the Averaged Perceptron.

Interested in creating an extension?

See the Contributing guide.

3.7 API Reference

3.7.1 Blob Classes

Wrappers for various units of text, including the main *TextBlob*, *Word*, and *WordList* classes. Example usage:

```
>>> from textblob import TextBlob
>>> b = TextBlob("Simple is better than complex.")
>>> b.tags
[(u'Simple', u'NN'), (u'is', u'VBZ'), (u'better', u'JJR'), (u'than', u'IN'), (u'complex',
↪ u'NN')]
>>> b.noun_phrases
WordList([u'simple'])
>>> b.words
WordList([u'Simple', u'is', u'better', u'than', u'complex'])
```

(continues on next page)

(continued from previous page)

```
>>> b.sentiment
(0.06666666666666667, 0.41904761904761906)
>>> b.words[0].synsets()[0]
Synset('simple.n.01')
```

Changed in version 0.8.0: These classes are now imported from `textblob` rather than `text.blob`.

class `textblob.blob.BaseBlob`(*text*, *tokenizer=None*, *pos_tagger=None*, *np_extractor=None*, *analyzer=None*, *parser=None*, *classifier=None*, *clean_html=False*)

An abstract base class that all textblob classes will inherit from. Includes words, POS tag, NP, and word count properties. Also includes basic dunder and string methods for making objects like Python strings.

Parameters

- **text** – A string.
- **tokenizer** – (optional) A tokenizer instance. If `None`, defaults to `WordTokenizer()`.
- **np_extractor** – (optional) An `NPE extractor` instance. If `None`, defaults to `FastNPE extractor()`.
- **pos_tagger** – (optional) A `Tagger` instance. If `None`, defaults to `NLTKTagger`.
- **analyzer** – (optional) A sentiment analyzer. If `None`, defaults to `PatternAnalyzer`.
- **parser** – A parser. If `None`, defaults to `PatternParser`.
- **classifier** – A classifier.

Changed in version 0.6.0: `clean_html` parameter deprecated, as it was in `NLTK`.

`classify()`

Classify the blob using the blob's `classifier`.

`correct()`

Attempt to correct the spelling of a blob.

Added in version 0.6.0.

Return type

`BaseBlob`

ends_with(*suffix*, *start=0*, *end=9223372036854775807*)

Returns `True` if the blob ends with the given suffix.

endswith(*suffix*, *start=0*, *end=9223372036854775807*)

Returns `True` if the blob ends with the given suffix.

find(*sub*, *start=0*, *end=9223372036854775807*)

Behaves like the built-in `str.find()` method. Returns an integer, the index of the first occurrence of the substring argument `sub` in the sub-string given by `[start:end]`.

format(**args*, ***kwargs*)

Perform a string formatting operation, like the built-in `str.format(*args, **kwargs)`. Returns a blob object.

index(*sub*, *start=0*, *end=9223372036854775807*)

Like `blob.find()` but raise `ValueError` when the substring is not found.

join(*iterable*)

Behaves like the built-in `str.join(iterable)` method, except returns a blob object.

Returns a blob which is the concatenation of the strings or blobs in the iterable.

lower()

Like `str.lower()`, returns new object with all lower-cased characters.

ngrams(*n*=3)

Return a list of n-grams (tuples of *n* successive words) for this blob.

Return type

List of *WordLists*

noun_phrases

Returns a list of noun phrases for this blob.

np_counts

Dictionary of noun phrase frequencies in this text.

parse(*parser*=None)

Parse the text.

Parameters

parser – (optional) A parser instance. If `None`, defaults to this blob's default parser.

Added in version 0.6.0.

polarity

Return the polarity score as a float within the range `[-1.0, 1.0]`

Return type

float

pos_tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

replace(*old*, *new*, *count*=9223372036854775807)

Return a new blob object with all occurrences of `old` replaced by `new`.

rfind(*sub*, *start*=0, *end*=9223372036854775807)

Behaves like the built-in `str.rfind()` method. Returns an integer, the index of the last (right-most) occurrence of the substring argument `sub` in the sub-sequence given by `[start:end]`.

rindex(*sub*, *start*=0, *end*=9223372036854775807)

Like `blob.rfind()` but raise `ValueError` when substring is not found.

sentiment

Return a tuple of form (polarity, subjectivity) where polarity is a float within the range [-1.0, 1.0] and subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

namedtuple of the form `Sentiment(polarity, subjectivity)`

sentiment_assessments

Return a tuple of form (polarity, subjectivity, assessments) where polarity is a float within the range [-1.0, 1.0], subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective, and assessments is a list of polarity and subjectivity scores for the assessed tokens.

Return type

namedtuple of the form `Sentiment(polarity, subjectivity, assessments)`

split(*sep*=None, *maxsplit*=9223372036854775807)

Behaves like the built-in `str.split()` except returns a `WordList`.

Return type

`WordList`

starts_with(*prefix*, *start*=0, *end*=9223372036854775807)

Returns True if the blob starts with the given prefix.

startswith(*prefix*, *start*=0, *end*=9223372036854775807)

Returns True if the blob starts with the given prefix.

strip(*chars*=None)

Behaves like the built-in `str.strip([chars])` method. Returns an object with leading and trailing whitespace removed.

subjectivity

Return the subjectivity score as a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

float

tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

title()

Returns a blob object with the text in title-case.

tokenize(*tokenizer=None*)

Return a list of tokens, using *tokenizer*.

Parameters

tokenizer – (optional) A tokenizer object. If None, defaults to this blob’s default tokenizer.

tokens

Return a list of tokens, using this blob’s tokenizer object (defaults to [WordTokenizer](#)).

upper()

Like str.upper(), returns new object with all upper-cased characters.

word_counts

Dictionary of word frequencies in this text.

words

Return a list of word tokens. This excludes punctuation characters. If you want to include punctuation characters, access the `tokens` property.

Returns

A [WordList](#) of word tokens.

```
class textblob.blob.Blobber(tokenizer=None, pos_tagger=None, np_extractor=None, analyzer=None,
                             parser=None, classifier=None)
```

A factory for TextBlobs that all share the same tagger, tokenizer, parser, classifier, and np_extractor.

Usage:

```
>>> from textblob import Blobber
>>> from textblob.taggers import NLTKTagger
>>> from textblob.tokenizers import SentenceTokenizer
>>> tb = Blobber(pos_tagger=NLTKTagger(), tokenizer=SentenceTokenizer())
>>> blob1 = tb("This is one blob.")
>>> blob2 = tb("This blob has the same tagger and tokenizer.")
>>> blob1.pos_tagger is blob2.pos_tagger
True
```

Parameters

- **tokenizer** – (optional) A tokenizer instance. If None, defaults to [WordTokenizer\(\)](#).
- **np_extractor** – (optional) An NPE extractor instance. If None, defaults to [FastNPE extractor\(\)](#).
- **pos_tagger** – (optional) A Tagger instance. If None, defaults to [NLTKTagger](#).
- **analyzer** – (optional) A sentiment analyzer. If None, defaults to [PatternAnalyzer](#).
- **parser** – A parser. If None, defaults to [PatternParser](#).
- **classifier** – A classifier.

Added in version 0.4.0.

```
class textblob.blob.Sentence(sentence, start_index=0, end_index=None, *args, **kwargs)
```

A sentence within a TextBlob. Inherits from [BaseBlob](#).

Parameters

- **sentence** – A string, the raw sentence.
- **start_index** – An int, the index where this sentence begins in a TextBlob. If not given, defaults to 0.
- **end_index** – An int, the index where this sentence ends in a TextBlob. If not given, defaults to the length of the sentence - 1.

classify()

Classify the blob using the blob's classifier.

correct()

Attempt to correct the spelling of a blob.

Added in version 0.6.0.

Return type

BaseBlob

property dict

The dict representation of this sentence.

end

The end index within a textBlob

end_index

The end index within a textBlob

ends_with(suffix, start=0, end=9223372036854775807)

Returns True if the blob ends with the given suffix.

endswith(suffix, start=0, end=9223372036854775807)

Returns True if the blob ends with the given suffix.

find(sub, start=0, end=9223372036854775807)

Behaves like the built-in str.find() method. Returns an integer, the index of the first occurrence of the substring argument sub in the sub-string given by [start:end].

format(*args, **kwargs)

Perform a string formatting operation, like the built-in str.format(*args, **kwargs). Returns a blob object.

index(sub, start=0, end=9223372036854775807)

Like blob.find() but raise ValueError when the substring is not found.

join(iterable)

Behaves like the built-in str.join(iterable) method, except returns a blob object.

Returns a blob which is the concatenation of the strings or blobs in the iterable.

lower()

Like str.lower(), returns new object with all lower-cased characters.

ngrams(n=3)

Return a list of n-grams (tuples of n successive words) for this blob.

Return type

List of *WordLists*

noun_phrases

Returns a list of noun phrases for this blob.

np_counts

Dictionary of noun phrase frequencies in this text.

parse(*parser=None*)

Parse the text.

Parameters

parser – (optional) A parser instance. If *None*, defaults to this blob's default parser.

Added in version 0.6.0.

polarity

Return the polarity score as a float within the range [-1.0, 1.0]

Return type

float

pos_tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

replace(*old, new, count=9223372036854775807*)

Return a new blob object with all occurrences of *old* replaced by *new*.

rfind(*sub, start=0, end=9223372036854775807*)

Behaves like the built-in `str.rfind()` method. Returns an integer, the index of the last (right-most) occurrence of the substring argument *sub* in the sub-sequence given by `[start:end]`.

rindex(*sub, start=0, end=9223372036854775807*)

Like `blob.rfind()` but raise `ValueError` when substring is not found.

sentiment

Return a tuple of form (polarity, subjectivity) where polarity is a float within the range [-1.0, 1.0] and subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

namedtuple of the form `Sentiment(polarity, subjectivity)`

sentiment_assessments

Return a tuple of form (polarity, subjectivity, assessments) where polarity is a float within the range [-1.0, 1.0], subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective, and assessments is a list of polarity and subjectivity scores for the assessed tokens.

Return typenamedtuple of the form `Sentiment(polarity, subjectivity, assessments)`**split**(*sep=None, maxsplit=9223372036854775807*)Behaves like the built-in `str.split()` except returns a `WordList`.**Return type***WordList***start**The start index within a `TextBlob`**start_index**The start index within a `TextBlob`**starts_with**(*prefix, start=0, end=9223372036854775807*)

Returns True if the blob starts with the given prefix.

startswith(*prefix, start=0, end=9223372036854775807*)

Returns True if the blob starts with the given prefix.

strip(*chars=None*)Behaves like the built-in `str.strip([chars])` method. Returns an object with leading and trailing whitespace removed.**subjectivity**

Return the subjectivity score as a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

float

tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

title()

Returns a blob object with the text in title-case.

tokenize(*tokenizer=None*)Return a list of tokens, using `tokenizer`.**Parameters****tokenizer** – (optional) A tokenizer object. If None, defaults to this blob's default tokenizer.

tokens

Return a list of tokens, using this blob's tokenizer object (defaults to [WordTokenizer](#)).

upper()

Like `str.upper()`, returns new object with all upper-cased characters.

word_counts

Dictionary of word frequencies in this text.

words

Return a list of word tokens. This excludes punctuation characters. If you want to include punctuation characters, access the `tokens` property.

Returns

A [WordList](#) of word tokens.

class `textblob.blob.TextBlob(text, tokenizer=None, pos_tagger=None, np_extractor=None, analyzer=None, parser=None, classifier=None, clean_html=False)`

A general text block, meant for larger bodies of text (esp. those containing sentences). Inherits from [BaseBlob](#).

Parameters

- **text** (*str*) – A string.
- **tokenizer** – (optional) A tokenizer instance. If `None`, defaults to [WordTokenizer\(\)](#).
- **np_extractor** – (optional) An `NPE extractor` instance. If `None`, defaults to [FastNPE extractor\(\)](#).
- **pos_tagger** – (optional) A `Tagger` instance. If `None`, defaults to [NLTKTagger](#).
- **analyzer** – (optional) A sentiment analyzer. If `None`, defaults to [PatternAnalyzer](#).
- **classifier** – (optional) A classifier.

classify()

Classify the blob using the blob's `classifier`.

correct()

Attempt to correct the spelling of a blob.

Added in version 0.6.0.

Return type

[BaseBlob](#)

ends_with(suffix, start=0, end=9223372036854775807)

Returns `True` if the blob ends with the given suffix.

endswith(suffix, start=0, end=9223372036854775807)

Returns `True` if the blob ends with the given suffix.

find(sub, start=0, end=9223372036854775807)

Behaves like the built-in `str.find()` method. Returns an integer, the index of the first occurrence of the substring argument `sub` in the sub-string given by `[start:end]`.

format(*args, **kwargs)

Perform a string formatting operation, like the built-in `str.format(*args, **kwargs)`. Returns a blob object.

index(*sub*, *start*=0, *end*=9223372036854775807)

Like `blob.find()` but raise `ValueError` when the substring is not found.

join(*iterable*)

Behaves like the built-in `str.join(iterable)` method, except returns a blob object.

Returns a blob which is the concatenation of the strings or blobs in the iterable.

property json

The json representation of this blob.

Changed in version 0.5.1: Made `json` a property instead of a method to restore backwards compatibility that was broken after version 0.4.0.

lower()

Like `str.lower()`, returns new object with all lower-cased characters.

ngrams(*n*=3)

Return a list of n-grams (tuples of *n* successive words) for this blob.

Return type

List of *WordLists*

noun_phrases

Returns a list of noun phrases for this blob.

np_counts

Dictionary of noun phrase frequencies in this text.

parse(*parser*=None)

Parse the text.

Parameters

parser – (optional) A parser instance. If `None`, defaults to this blob's default parser.

Added in version 0.6.0.

polarity

Return the polarity score as a float within the range [-1.0, 1.0]

Return type

float

pos_tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

property raw_sentences

List of strings, the raw sentences in the blob.

replace(*old*, *new*, *count*=9223372036854775807)

Return a new blob object with all occurrences of *old* replaced by *new*.

rfind(*sub*, *start*=0, *end*=9223372036854775807)

Behaves like the built-in `str.rfind()` method. Returns an integer, the index of the last (right-most) occurrence of the substring argument *sub* in the sub-sequence given by [*start*:*end*].

rindex(*sub*, *start*=0, *end*=9223372036854775807)

Like `blob.rfind()` but raise `ValueError` when substring is not found.

sentences

Return list of [Sentence](#) objects.

sentiment

Return a tuple of form (polarity, subjectivity) where polarity is a float within the range [-1.0, 1.0] and subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

namedtuple of the form `Sentiment(polarity, subjectivity)`

sentiment_assessments

Return a tuple of form (polarity, subjectivity, assessments) where polarity is a float within the range [-1.0, 1.0], subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective, and assessments is a list of polarity and subjectivity scores for the assessed tokens.

Return type

namedtuple of the form `Sentiment(polarity, subjectivity, assessments)`

property serialized

Returns a list of each sentence's dict representation.

split(*sep*=None, *maxsplit*=9223372036854775807)

Behaves like the built-in `str.split()` except returns a `WordList`.

Return type

[WordList](#)

starts_with(*prefix*, *start*=0, *end*=9223372036854775807)

Returns True if the blob starts with the given prefix.

startswith(*prefix*, *start*=0, *end*=9223372036854775807)

Returns True if the blob starts with the given prefix.

strip(*chars*=None)

Behaves like the built-in `str.strip([chars])` method. Returns an object with leading and trailing whitespace removed.

subjectivity

Return the subjectivity score as a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is very subjective.

Return type

float

tags

Returns an list of tuples of the form (word, POS tag).

Example:

```
[
    ("At", "IN"),
    ("eight", "CD"),
    ("o'clock", "JJ"),
    ("on", "IN"),
    ("Thursday", "NNP"),
    ("morning", "NN"),
]
```

Return type

list of tuples

title()

Returns a blob object with the text in title-case.

to_json(*args, **kwargs)

Return a json representation (str) of this blob. Takes the same arguments as json.dumps.

Added in version 0.5.1.

tokenize(tokenizer=None)

Return a list of tokens, using `tokenizer`.

Parameters

tokenizer – (optional) A tokenizer object. If None, defaults to this blob's default tokenizer.

tokens

Return a list of tokens, using this blob's tokenizer object (defaults to [WordTokenizer](#)).

upper()

Like str.upper(), returns new object with all upper-cased characters.

word_counts

Dictionary of word frequencies in this text.

words

Return a list of word tokens. This excludes punctuation characters. If you want to include punctuation characters, access the `tokens` property.

Returns

A [WordList](#) of word tokens.

class textblob.blob.**Word**(string, pos_tag=None)

A simple word representation. Includes methods for inflection, and WordNet integration.

capitalize()

Return a capitalized version of the string.

More specifically, make the first character have upper case and the rest lower case.

casefold()

Return a version of the string suitable for caseless comparisons.

center(*width*, *fillchar*=' ', / (*Positional-only parameter separator (PEP 570)*))

Return a centered string of length *width*.

Padding is done using the specified fill character (default is a space).

correct()

Correct the spelling of the word. Returns the word with the highest confidence using the spelling corrector.

Added in version 0.6.0.

count()

Return the number of non-overlapping occurrences of substring *sub* in string *S*[*start*:*end*].

Optional arguments *start* and *end* are interpreted as in slice notation.

define(*pos*=None)

Return a list of definitions for this word. Each definition corresponds to a synset for this word.

Parameters

pos – A part-of-speech tag to filter upon. If *None*, definitions for all parts of speech will be loaded.

Return type

List of strings

Added in version 0.7.0.

definitions

The list of definitions for this word. Each definition corresponds to a synset.

Added in version 0.7.0.

encode(*encoding*='utf-8', *errors*='strict')

Encode the string using the codec registered for encoding.

encoding

The encoding in which to encode the string.

errors

The error handling scheme to use for encoding errors. The default is 'strict' meaning that encoding errors raise a `UnicodeEncodeError`. Other possible values are 'ignore', 'replace' and 'xmlcharrefreplace' as well as any other name registered with `codecs.register_error` that can handle `UnicodeEncodeErrors`.

endswith()

Return True if the string ends with the specified suffix, False otherwise.

suffix

A string or a tuple of strings to try.

start

Optional start position. Default: start of the string.

end

Optional stop position. Default: end of the string.

expandtabs(*tabsize*=8)

Return a copy where all tab characters are expanded using spaces.

If *tabsize* is not given, a tab size of 8 characters is assumed.

find()

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation. Return -1 on failure.

format(*args, **kwargs)

Return a formatted version of the string, using substitutions from args and kwargs. The substitutions are identified by braces ('{' and '}').

format_map(mapping, /)

Return a formatted version of the string, using substitutions from mapping. The substitutions are identified by braces ('{' and '}').

get_synsets(pos=None)

Return a list of Synset objects for this word.

Parameters

pos – A part-of-speech tag to filter upon. If None, all synsets for all parts of speech will be loaded.

Return type

list of Synsets

Added in version 0.7.0.

index()

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation. Raises ValueError when the substring is not found.

isalnum()

Return True if the string is an alpha-numeric string, False otherwise.

A string is alpha-numeric if all characters in the string are alpha-numeric and there is at least one character in the string.

isalpha()

Return True if the string is an alphabetic string, False otherwise.

A string is alphabetic if all characters in the string are alphabetic and there is at least one character in the string.

isascii()

Return True if all characters in the string are ASCII, False otherwise.

ASCII characters have code points in the range U+0000-U+007F. Empty string is ASCII too.

isdecimal()

Return True if the string is a decimal string, False otherwise.

A string is a decimal string if all characters in the string are decimal and there is at least one character in the string.

isdigit()

Return True if the string is a digit string, False otherwise.

A string is a digit string if all characters in the string are digits and there is at least one character in the string.

isidentifier()

Return True if the string is a valid Python identifier, False otherwise.

Call keyword.iskeyword(s) to test whether string s is a reserved identifier, such as “def” or “class”.

islower()

Return True if the string is a lowercase string, False otherwise.

A string is lowercase if all cased characters in the string are lowercase and there is at least one cased character in the string.

isnumeric()

Return True if the string is a numeric string, False otherwise.

A string is numeric if all characters in the string are numeric and there is at least one character in the string.

isprintable()

Return True if all characters in the string are printable, False otherwise.

A character is printable if repr() may use it in its output.

isspace()

Return True if the string is a whitespace string, False otherwise.

A string is whitespace if all characters in the string are whitespace and there is at least one character in the string.

istitle()

Return True if the string is a title-cased string, False otherwise.

In a title-cased string, upper- and title-case characters may only follow uncased characters and lowercase characters only cased ones.

isupper()

Return True if the string is an uppercase string, False otherwise.

A string is uppercase if all cased characters in the string are uppercase and there is at least one cased character in the string.

join(iterable, /)

Concatenate any number of strings.

The string whose method is called is inserted in between each given string. The result is returned as a new string.

Example: `'.'.join(['ab', 'pq', 'rs']) -> 'ab.pq.rs'`

lemma

Return the lemma of this word using Wordnet’s morphy function.

lemmatize(pos=None)

Return the lemma for a word using WordNet’s morphy function.

Parameters

pos – Part of speech to filter upon. If None, defaults to `_wordnet.NOUN`.

Added in version 0.8.1.

ljust(width, fillchar=' ', /)

Return a left-justified string of length width.

Padding is done using the specified fill character (default is a space).

lower()

Return a copy of the string converted to lowercase.

lstrip(*chars=None, /*)

Return a copy of the string with leading whitespace removed.

If *chars* is given and not *None*, remove characters in *chars* instead.

static maketrans()

Return a translation table usable for *str.translate()*.

If there is only one argument, it must be a dictionary mapping Unicode ordinals (integers) or characters to Unicode ordinals, strings or *None*. Character keys will be then converted to ordinals. If there are two arguments, they must be strings of equal length, and in the resulting dictionary, each character in *x* will be mapped to the character at the same position in *y*. If there is a third argument, it must be a string, whose characters will be mapped to *None* in the result.

partition(*sep, /*)

Partition the string into three parts using the given separator.

This will search for the separator in the string. If the separator is found, returns a 3-tuple containing the part before the separator, the separator itself, and the part after it.

If the separator is not found, returns a 3-tuple containing the original string and two empty strings.

pluralize()

Return the plural version of the word as a string.

removeprefix(*prefix, /*)

Return a *str* with the given prefix string removed if present.

If the string starts with the prefix string, return *string[len(prefix):]*. Otherwise, return a copy of the original string.

removesuffix(*suffix, /*)

Return a *str* with the given suffix string removed if present.

If the string ends with the suffix string and that suffix is not empty, return *string[:-len(suffix)]*. Otherwise, return a copy of the original string.

replace(*old, new, /, count=-1*)

Return a copy with all occurrences of substring *old* replaced by *new*.

count

Maximum number of occurrences to replace. -1 (the default value) means replace all occurrences.

If the optional argument *count* is given, only the first *count* occurrences are replaced.

rfind()

Return the highest index in *S* where substring *sub* is found, such that *sub* is contained within *S[start:end]*.

Optional arguments *start* and *end* are interpreted as in slice notation. Return -1 on failure.

rindex()

Return the highest index in *S* where substring *sub* is found, such that *sub* is contained within *S[start:end]*.

Optional arguments *start* and *end* are interpreted as in slice notation. Raises *ValueError* when the substring is not found.

rjust(width, fillchar=' ', l)

Return a right-justified string of length width.

Padding is done using the specified fill character (default is a space).

rpartition(sep, l)

Partition the string into three parts using the given separator.

This will search for the separator in the string, starting at the end. If the separator is found, returns a 3-tuple containing the part before the separator, the separator itself, and the part after it.

If the separator is not found, returns a 3-tuple containing two empty strings and the original string.

rsplit(sep=None, maxsplit=-1)

Return a list of the substrings in the string, using sep as the separator string.

sep

The separator used to split the string.

When set to None (the default value), will split on any whitespace character (including n r t f and spaces) and will discard empty strings from the result.

maxsplit

Maximum number of splits. -1 (the default value) means no limit.

Splitting starts at the end of the string and works to the front.

rstrip(chars=None, l)

Return a copy of the string with trailing whitespace removed.

If chars is given and not None, remove characters in chars instead.

singularize()

Return the singular version of the word as a string.

spellcheck()

Return a list of (word, confidence) tuples of spelling corrections.

Based on: Peter Norvig, “How to Write a Spelling Corrector” (<http://norvig.com/spell-correct.html>) as implemented in the pattern library.

Added in version 0.6.0.

split(sep=None, maxsplit=-1)

Return a list of the substrings in the string, using sep as the separator string.

sep

The separator used to split the string.

When set to None (the default value), will split on any whitespace character (including n r t f and spaces) and will discard empty strings from the result.

maxsplit

Maximum number of splits. -1 (the default value) means no limit.

Splitting starts at the front of the string and works to the end.

Note, str.split() is mainly useful for data that has been intentionally delimited. With natural text that includes punctuation, consider using the regular expression module.

splitlines(keepends=False)

Return a list of the lines in the string, breaking at line boundaries.

Line breaks are not included in the resulting list unless keepends is given and true.

startswith()

Return True if the string starts with the specified prefix, False otherwise.

prefix

A string or a tuple of strings to try.

start

Optional start position. Default: start of the string.

end

Optional stop position. Default: end of the string.

stem(*stemmer=<PorterStemmer>*)

Stem a word using various NLTK stemmers. (Default: Porter Stemmer)

Added in version 0.12.0.

strip(*chars=None, /*)

Return a copy of the string with leading and trailing whitespace removed.

If chars is given and not None, remove characters in chars instead.

swapcase()

Convert uppercase characters to lowercase and lowercase characters to uppercase.

synsets

The list of Synset objects for this Word.

Return type

list of Synsets

Added in version 0.7.0.

title()

Return a version of the string where each word is titlecased.

More specifically, words start with uppercased characters and all remaining cased characters have lower case.

translate(*table, /*)

Replace each character in the string using the given translation table.

table

Translation table, which must be a mapping of Unicode ordinals to Unicode ordinals, strings, or None.

The table must implement lookup/indexing via `__getitem__`, for instance a dictionary or list. If this operation raises `LookupError`, the character is left untouched. Characters mapped to None are deleted.

upper()

Return a copy of the string converted to uppercase.

zfill(*width, /*)

Pad a numeric string with zeros on the left, to fill a field of the given width.

The string is never truncated.

class textblob.blob.**WordList**(*collection*)

A list-like collection of words.

append(*obj*)

Append an object to end. If the object is a string, appends a *Word* object.

clear()

Remove all items from list.

copy()

Return a shallow copy of the list.

count(*strg*, *case_sensitive=False*, **args*, *kwargs*)**

Get the count of a word or phrase *s* within this WordList.

Parameters

- **strg** – The string to count.
- **case_sensitive** – A boolean, whether or not the search is case-sensitive.

extend(*iterable*)

Extend WordList by appending elements from *iterable*. If an element is a string, appends a *Word* object.

index(*value*, *start=0*, *stop=9223372036854775807*, *l*)

Return first index of value.

Raises ValueError if the value is not present.

insert(*index*, *object*, *l*)

Insert object before index.

lemmatize()

Return the lemma of each word in this WordList.

lower()

Return a new WordList with each word lower-cased.

pluralize()

Return the plural version of each word in this WordList.

pop(*index=-1*, *l*)

Remove and return item at index (default last).

Raises IndexError if list is empty or index is out of range.

remove(*value*, *l*)

Remove first occurrence of value.

Raises ValueError if the value is not present.

reverse()

Reverse *IN PLACE*.

singularize()

Return the single version of each word in this WordList.

sort(* (*Keyword-only parameters separator (PEP 3102)*), *key=None*, *reverse=False*)

Sort the list in ascending order and return None.

The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values.

The reverse flag can be set to sort in descending order.

stem(*args, **kwargs)

Return the stem for each word in this WordList.

upper()

Return a new WordList with each word upper-cased.

3.7.2 Base Classes

Abstract base classes for models (taggers, noun phrase extractors, etc.) which define the interface for descendant classes.

Changed in version 0.7.0: All base classes are defined in the same module, `textblob.base`.

class `textblob.base.BaseNPExtractor`

Abstract base class from which all NPExtractor classes inherit. Descendant classes must implement an `extract(text)` method that returns a list of noun phrases as strings.

abstractmethod `extract(text: str) → list[str]`

Return a list of noun phrases (strings) for a body of text.

class `textblob.base.BaseParser`

Abstract parser class from which all parsers inherit from. All descendants must implement a `parse()` method.

abstractmethod `parse(text: AnyStr)`

Parses the text.

class `textblob.base.BaseSentimentAnalyzer`

Abstract base class from which all sentiment analyzers inherit. Should implement an `analyze(text)` method which returns either the results of analysis.

abstractmethod `analyze(text) → Any`

Return the result of of analysis. Typically returns either a tuple, float, or dictionary.

class `textblob.base.BaseTagger`

Abstract tagger class from which all taggers inherit from. All descendants must implement a `tag()` method.

abstractmethod `tag(text: str, tokenize=True) → list[tuple[str, str]]`

Return a list of tuples of the form (word, tag) for a given set of text or BaseBlob instance.

class `textblob.base.BaseTokenizer`

Abstract base class from which all Tokenizer classes inherit. Descendant classes must implement a `tokenize(text)` method that returns a list of noun phrases as strings.

itokenize(text: str, *args, **kwargs)

Return a generator that generates tokens “on-demand”.

Added in version 0.6.0.

Return type

generator

abstractmethod `tokenize(text: str) → list[str]`

Return a list of tokens (strings) for a body of text.

Return type

list

3.7.3 Tokenizers

Various tokenizer implementations.

Added in version 0.4.0.

class textblob.tokenizers.**SentenceTokenizer**

NLTK's sentence tokenizer (currently PunktSentenceTokenizer). Uses an unsupervised algorithm to build a model for abbreviation words, collocations, and words that start sentences, then uses that to find sentence boundaries.

itokenize(*text: str, *args, **kwargs*)

Return a generator that generates tokens "on-demand".

Added in version 0.6.0.

Return type

generator

span_tokenize(*s: str*) → Iterator[tuple[int, int]]

Identify the tokens using integer offsets (*start_i*, *end_i*), where *s*[*start_i*:*end_i*] is the corresponding token.

Return type

Iterator[Tuple[int, int]]

span_tokenize_sents(*strings: list[str]*) → Iterator[list[tuple[int, int]]]

Apply *self.span_tokenize()* to each element of *strings*. I.e.:

return [self.span_tokenize(s) for s in strings]

Yield

List[Tuple[int, int]]

tokenize(*text*)

Return a list of sentences.

tokenize_sents(*strings: list[str]*) → list[list[str]]

Apply *self.tokenize()* to each element of *strings*. I.e.:

return [self.tokenize(s) for s in strings]

Return type

List[List[str]]

class textblob.tokenizers.**WordTokenizer**

NLTK's recommended word tokenizer (currently the TreeBankTokenizer). Uses regular expressions to tokenize text. Assumes text has already been segmented into sentences.

Performs the following steps:

- split standard contractions, e.g. don't -> do n't
- split commas and single quotes
- separate periods that appear at the end of line

itokenize(*text: str, *args, **kwargs*)

Return a generator that generates tokens "on-demand".

Added in version 0.6.0.

Return type

generator

span_tokenize(*s: str*) → Iterator[tuple[int, int]]

Identify the tokens using integer offsets (*start_i*, *end_i*), where *s*[*start_i*:*end_i*] is the corresponding token.

Return type

Iterator[Tuple[int, int]]

span_tokenize_sents(*strings: list[str]*) → Iterator[list[tuple[int, int]]]

Apply `self.span_tokenize()` to each element of *strings*. I.e.:

return [self.span_tokenize(s) for s in strings]

Yield

List[Tuple[int, int]]

tokenize(*text*, *include_punc=True*)

Return a list of word tokens.

Parameters

- **text** – string of text.
- **include_punc** – (optional) whether to include punctuation as separate tokens. Default to `True`.

tokenize_sents(*strings: list[str]*) → list[list[str]]

Apply `self.tokenize()` to each element of *strings*. I.e.:

return [self.tokenize(s) for s in strings]

Return type

List[List[str]]

textblob.tokenizers.sent_tokenize(*text: str*, **args*, ***kwargs*)

Convenience function for tokenizing sentences

textblob.tokenizers.word_tokenize(*text*, *include_punc=True*, **args*, ***kwargs*)

Convenience function for tokenizing text into words.

NOTE: NLTK's word tokenizer expects sentences as input, so the text will be tokenized to sentences before being tokenized to words.

3.7.4 POS Taggers

Parts-of-speech tagger implementations.

class textblob.en.taggers.NLTKTagger

Tagger that uses NLTK's standard TreeBank tagger. NOTE: Requires numpy. Not yet supported with PyPy.

tag(*text*)

Tag a string or BaseBlob.

class textblob.en.taggers.PatternTagger

Tagger that uses the implementation in Tom de Smedt's pattern library (<http://www.clips.ua.ac.be/pattern>).

tag(*text*, *tokenize=True*)

Tag a string or BaseBlob.

3.7.5 Noun Phrase Extractors

Various noun phrase extractors.

class textblob.en.np_extractors.**ChunkParser**

accuracy(*gold*)

Score the accuracy of the chunker against the gold standard. Remove the chunking the gold standard text, rechunk it using the chunker, and return a `ChunkScore` object reflecting the performance of this chunk parser.

Parameters

gold (*list*(*Tree*)) – The list of chunked sentences to score the chunker on.

Return type

`ChunkScore`

evaluate(***kwargs*)

@deprecated: Use `accuracy(gold)` instead.

grammar()

Returns

The grammar used by this parser.

parse(*tokens*)

Return the parse tree for the sentence.

parse_all(*sent*, **args*, ***kwargs*)

Return type

`list`(*Tree*)

parse_one(*sent*, **args*, ***kwargs*)

Return type

`Tree` or `None`

parse_sents(*sents*, **args*, ***kwargs*)

Apply `self.parse()` to each element of `sents`. :rtype: `iter(iter(Tree))`

train()

Train the Chunker on the ConLL-2000 corpus.

class textblob.en.np_extractors.**ConllExtractor**(*parser=None*)

A noun phrase extractor that uses chunk parsing trained with the ConLL-2000 training corpus.

extract(*text*)

Return a list of noun phrases (strings) for body of text.

class textblob.en.np_extractors.**FastNPExtractor**

A fast and simple noun phrase extractor.

Credit to Shlomi Babluk. Link to original blog post:

<http://thetokenizer.com/2013/05/09/efficient-way-to-extract-the-main-topics-of-a-sentence/>

extract(*text*)

Return a list of noun phrases (strings) for body of text.

3.7.6 Sentiment Analyzers

Sentiment analysis implementations.

Added in version 0.5.0.

```
class textblob.en.sentiments.NaiveBayesAnalyzer(feature_extractor=<function  
                                              _default_feature_extractor>)
```

Naive Bayes analyzer that is trained on a dataset of movie reviews. Returns results as a named tuple of the form: `Sentiment(classification, p_pos, p_neg)`

Parameters

feature_extractor (*callable*) – Function that returns a dictionary of features, given a list of words.

RETURN_TYPE

Return type declaration

alias of `Sentiment`

analyze(*text*)

Return the sentiment as a named tuple of the form: `Sentiment(classification, p_pos, p_neg)`

train()

Train the Naive Bayes classifier on the movie review corpus.

```
class textblob.en.sentiments.PatternAnalyzer
```

Sentiment analyzer that uses the same implementation as the pattern library. Returns results as a named tuple of the form:

`Sentiment(polarity, subjectivity, [assessments])`

where `[assessments]` is a list of the assessed tokens and their polarity and subjectivity scores

RETURN_TYPE

alias of `Sentiment`

analyze(*text*, *keep_assessments=False*)

Return the sentiment as a named tuple of the form: `Sentiment(polarity, subjectivity, [assessments])`.

3.7.7 Parsers

Various parser implementations.

Added in version 0.6.0.

```
class textblob.en.parsers.PatternParser
```

Parser that uses the implementation in Tom de Smedt's pattern library. <http://www.clips.ua.ac.be/pages/pattern-en#parser>

parse(*text*)

Parses the text.

3.7.8 Classifiers

Various classifier implementations. Also includes basic feature extractor methods.

Example Usage:

```

>>> from textblob import TextBlob
>>> from textblob.classifiers import NaiveBayesClassifier
>>> train = [
...     ('I love this sandwich.', 'pos'),
...     ('This is an amazing place!', 'pos'),
...     ('I feel very good about these beers.', 'pos'),
...     ('I do not like this restaurant', 'neg'),
...     ('I am tired of this stuff.', 'neg'),
...     ('I can't deal with this', 'neg'),
...     ('My boss is horrible.', 'neg')
... ]
>>> cl = NaiveBayesClassifier(train)
>>> cl.classify("I feel amazing!")
'pos'
>>> blob = TextBlob("The beer is good. But the hangover is horrible.", classifier=cl)
>>> for s in blob.sentences:
...     print(s)
...     print(s.classify())
...
The beer is good.
pos
But the hangover is horrible.
neg

```

Added in version 0.6.0.

class textblob.classifiers.**BaseClassifier**(*train_set*, *feature_extractor*=<function basic_extractor>, *format*=None, **kwargs)

Abstract classifier class from which all classifiers inherit. At a minimum, descendant classes must implement a `classify` method and have a `classifier` property.

Parameters

- **train_set** – The training set, either a list of tuples of the form (text, classification) or a file-like object. text may be either a string or an iterable.
- **feature_extractor** (*callable*) – A feature extractor function that takes one or two arguments: document and train_set.
- **format** (*str*) – If train_set is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.
- **kwargs** – Additional keyword arguments are passed to the constructor of the *Format* class used to read the data. Only applies when a file-like object is passed as train_set.

Added in version 0.6.0.

classifier

The classifier object.

classify(*text*)

Classifies a string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

labels()

Returns an iterable containing the possible labels.

train(*labeled_featureset*)

Trains the classifier.

class textblob.classifiers.DecisionTreeClassifier(*train_set*, *feature_extractor*=<function *basic_extractor*>, *format*=None, *kwargs*)**

A classifier based on the decision tree algorithm, as implemented in NLTK.

Parameters

- **train_set** – The training set, either a list of tuples of the form (*text*, *classification*) or a filename. *text* may be either a string or an iterable.
- **feature_extractor** – A feature extractor function that takes one or two arguments: document and *train_set*.
- **format** – If *train_set* is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

Added in version 0.6.2.

accuracy(*test_set*, *format*=None)

Compute the accuracy on a test set.

Parameters

- **test_set** – A list of tuples of the form (*text*, *label*), or a file pointer.
- **format** – If *test_set* is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

classifier

The classifier.

classify(*text*)

Classifies the text.

Parameters

text (*str*) – A string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

labels()

Return an iterable of possible labels.

nltk_class

alias of DecisionTreeClassifier

pprint(args*, ***kwargs*)**

Return a string containing a pretty-printed version of this decision tree. Each line in the string corresponds to a single decision tree node or leaf, and indentation is used to display the structure of the tree.

Return type

str

pretty_format(*args, **kwargs)

Return a string containing a pretty-printed version of this decision tree. Each line in the string corresponds to a single decision tree node or leaf, and indentation is used to display the structure of the tree.

Return type

str

pseudocode(*args, **kwargs)

Return a string representation of this decision tree that expresses the decisions it makes as a nested set of pseudocode if statements.

Return type

str

train(*args, **kwargs)

Train the classifier with a labeled feature set and return the classifier. Takes the same arguments as the wrapped NLTK class. This method is implicitly called when calling `classify` or `accuracy` methods and is included only to allow passing in arguments to the `train` method of the wrapped NLTK class.

Added in version 0.6.2.

Return type

A classifier

update(new_data, *args, **kwargs)

Update the classifier with new training data and re-trains the classifier.

Parameters

new_data – New data as a list of tuples of the form (text, label).

class textblob.classifiers.**MaxEntClassifier**(train_set, feature_extractor=<function basic_extractor>, format=None, **kwargs)

A maximum entropy classifier (also known as a “conditional exponential classifier”). This classifier is parameterized by a set of “weights”, which are used to combine the joint-features that are generated from a featureset by an “encoding”. In particular, the encoding maps each (featureset, label) pair to a vector. The probability of each label is then computed using the following equation:

$$\text{prob}(\text{fs}|\text{label}) = \frac{\text{dotprod}(\text{weights}, \text{encode}(\text{fs}, \text{label}))}{\sum(\text{dotprod}(\text{weights}, \text{encode}(\text{fs}, \text{l})) \text{ for } \text{l in labels})}$$

Where `dotprod` is the dot product:

$$\text{dotprod}(\text{a}, \text{b}) = \sum(\text{x} * \text{y} \text{ for } (\text{x}, \text{y}) \text{ in zip}(\text{a}, \text{b}))$$

accuracy(test_set, format=None)

Compute the accuracy on a test set.

Parameters

- **test_set** – A list of tuples of the form (text, label), or a file pointer.
- **format** – If `test_set` is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

classifier

The classifier.

classify(*text*)

Classifies the text.

Parameters

text (*str*) – A string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

labels()

Return an iterable of possible labels.

nltk_class

alias of MaxentClassifier

prob_classify(*text*)

Return the label probability distribution for classifying a string of text.

Example:

```
>>> classifier = MaxEntClassifier(train_data)
>>> prob_dist = classifier.prob_classify("I feel happy this morning.")
>>> prob_dist.max()
'positive'
>>> prob_dist.prob("positive")
0.7
```

Return type

nltk.probability.DictionaryProbDist

train(*args, **kwargs)

Train the classifier with a labeled feature set and return the classifier. Takes the same arguments as the wrapped NLTK class. This method is implicitly called when calling `classify` or `accuracy` methods and is included only to allow passing in arguments to the `train` method of the wrapped NLTK class.

Added in version 0.6.2.

Return type

A classifier

update(*new_data*, *args, **kwargs)

Update the classifier with new training data and re-trains the classifier.

Parameters

new_data – New data as a list of tuples of the form (*text*, *label*).

class textblob.classifiers.**NLTKClassifier**(*train_set*, *feature_extractor*=<function *basic_extractor*>, *format*=None, **kwargs)

An abstract class that wraps around the `nltk.classify` module.

Expects that descendant classes include a class variable `nltk_class` which is the class in the `nltk.classify` module to be wrapped.

Example:

```
class MyClassifier(NLTKClassifier):
    nltk_class = nltk.classify.svm.SvmClassifier
```

accuracy(*test_set*, *format=None*)

Compute the accuracy on a test set.

Parameters

- **test_set** – A list of tuples of the form (text, label), or a file pointer.
- **format** – If test_set is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

classifier

The classifier.

classify(*text*)

Classifies the text.

Parameters

text (*str*) – A string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

labels()

Return an iterable of possible labels.

nltk_class = None

The NLTK class to be wrapped. Must be a class within nltk.classify

train(*args, **kwargs)

Train the classifier with a labeled feature set and return the classifier. Takes the same arguments as the wrapped NLTK class. This method is implicitly called when calling `classify` or `accuracy` methods and is included only to allow passing in arguments to the `train` method of the wrapped NLTK class.

Added in version 0.6.2.

Return type

A classifier

update(*new_data*, *args, **kwargs)

Update the classifier with new training data and re-trains the classifier.

Parameters

new_data – New data as a list of tuples of the form (text, label).

```
class textblob.classifiers.NaiveBayesClassifier(train_set, feature_extractor=<function
                                             basic_extractor>, format=None, **kwargs)
```

A classifier based on the Naive Bayes algorithm, as implemented in NLTK.

Parameters

- **train_set** – The training set, either a list of tuples of the form (text, classification) or a filename. text may be either a string or an iterable.
- **feature_extractor** – A feature extractor function that takes one or two arguments: document and train_set.

- **format** – If `train_set` is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

Added in version 0.6.0.

accuracy(*test_set*, *format=None*)

Compute the accuracy on a test set.

Parameters

- **test_set** – A list of tuples of the form (`text`, `label`), or a file pointer.
- **format** – If `test_set` is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

classifier

The classifier.

classify(*text*)

Classifies the text.

Parameters

text (*str*) – A string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

informative_features(*args, **kwargs)

Return the most informative features as a list of tuples of the form (`feature_name`, `feature_value`).

Return type

list

labels()

Return an iterable of possible labels.

nltk_class

alias of `NaiveBayesClassifier`

prob_classify(*text*)

Return the label probability distribution for classifying a string of text.

Example:

```
>>> classifier = NaiveBayesClassifier(train_data)
>>> prob_dist = classifier.prob_classify("I feel happy this morning.")
>>> prob_dist.max()
'positive'
>>> prob_dist.prob("positive")
0.7
```

Return type

`nltk.probability.DictionaryProbDist`

show_informative_features(*args, **kwargs)

Displays a listing of the most informative features for this classifier.

Return type

None

train(*args, **kwargs)

Train the classifier with a labeled feature set and return the classifier. Takes the same arguments as the wrapped NLTK class. This method is implicitly called when calling `classify` or `accuracy` methods and is included only to allow passing in arguments to the `train` method of the wrapped NLTK class.

Added in version 0.6.2.

Return type

A classifier

update(new_data, *args, **kwargs)

Update the classifier with new training data and re-trains the classifier.

Parameters

new_data – New data as a list of tuples of the form (text, label).

```
class textblob.classifiers.PositiveNaiveBayesClassifier(positive_set, unlabeled_set,
                                                    feature_extractor=<function
                                                    contains_extractor>,
                                                    positive_prob_prior=0.5, **kwargs)
```

A variant of the Naive Bayes Classifier that performs binary classification with partially-labeled training sets, i.e. when only one class is labeled and the other is not. Assuming a prior distribution on the two labels, uses the unlabeled set to estimate the frequencies of the features.

Example usage:

```
>>> from text.classifiers import PositiveNaiveBayesClassifier
>>> sports_sentences = ['The team dominated the game',
...                     'They lost the ball',
...                     'The game was intense',
...                     'The goalkeeper caught the ball',
...                     'The other team controlled the ball']
>>> various_sentences = ['The President did not comment',
...                      'I lost the keys',
...                      'The team won the game',
...                      'Sara has two kids',
...                      'The ball went off the court',
...                      'They had the ball for the whole game',
...                      'The show is over']
>>> classifier = PositiveNaiveBayesClassifier(positive_set=sports_sentences,
...                                           unlabeled_set=various_sentences)
>>> classifier.classify("My team lost the game")
True
>>> classifier.classify("And now for something completely different.")
False
```

Parameters

- **positive_set** – A collection of strings that have the positive label.
- **unlabeled_set** – A collection of unlabeled strings.

- **feature_extractor** – A feature extractor function.
- **positive_prob_prior** – A prior estimate of the probability of the label True.

Added in version 0.7.0.

accuracy(*test_set*, *format=None*)

Compute the accuracy on a test set.

Parameters

- **test_set** – A list of tuples of the form (*text*, *label*), or a file pointer.
- **format** – If *test_set* is a filename, the file format, e.g. "csv" or "json". If None, will attempt to detect the file format.

classifier

The classifier.

classify(*text*)

Classifies the text.

Parameters

text (*str*) – A string of text.

extract_features(*text*)

Extracts features from a body of text.

Return type

dictionary of features

labels()

Return an iterable of possible labels.

nltk_class

alias of `PositiveNaiveBayesClassifier`

train(**args*, ***kwargs*)

Train the classifier with a labeled and unlabeled feature sets and return the classifier. Takes the same arguments as the wrapped NLTK class. This method is implicitly called when calling `classify` or `accuracy` methods and is included only to allow passing in arguments to the `train` method of the wrapped NLTK class.

Return type

A classifier

update(*new_positive_data=None*, *new_unlabeled_data=None*, *positive_prob_prior=0.5*, **args*, ***kwargs*)

Update the classifier with new data and re-trains the classifier.

Parameters

- **new_positive_data** – List of new, labeled strings.
- **new_unlabeled_data** – List of new, unlabeled strings.

`textblob.classifiers.basic_extractor`(*document*, *train_set*)

A basic document feature extractor that returns a dict indicating what words in *train_set* are contained in *document*.

Parameters

- **document** – The text to extract features from. Can be a string or an iterable.

- **train_set** (*List*) – Training data set, a list of tuples of the form (words, label) OR an iterable of strings.

`textblob.classifiers.contains_extractor(document)`

A basic document feature extractor that returns a dict of words that the document contains.

3.7.9 Blobber

class `textblob.blob.Blobber`(*tokenizer=None, pos_tagger=None, np_extractor=None, analyzer=None, parser=None, classifier=None*)

A factory for TextBlobs that all share the same tagger, tokenizer, parser, classifier, and np_extractor.

Usage:

```
>>> from textblob import Blobber
>>> from textblob.taggers import NLTKTagger
>>> from textblob.tokenizers import SentenceTokenizer
>>> tb = Blobber(pos_tagger=NLTKTagger(), tokenizer=SentenceTokenizer())
>>> blob1 = tb("This is one blob.")
>>> blob2 = tb("This blob has the same tagger and tokenizer.")
>>> blob1.pos_tagger is blob2.pos_tagger
True
```

Parameters

- **tokenizer** – (optional) A tokenizer instance. If *None*, defaults to [WordTokenizer\(\)](#).
- **np_extractor** – (optional) An *NPExtractor* instance. If *None*, defaults to [FastNPExtractor\(\)](#).
- **pos_tagger** – (optional) A *Tagger* instance. If *None*, defaults to [NLTKTagger](#).
- **analyzer** – (optional) A sentiment analyzer. If *None*, defaults to [PatternAnalyzer](#).
- **parser** – A parser. If *None*, defaults to [PatternParser](#).
- **classifier** – A classifier.

Added in version 0.4.0.

__call__(*text*)

Return a new *TextBlob* object with this *Blobber*’s *np_extractor*, *pos_tagger*, *tokenizer*, *analyzer*, and *classifier*.

Returns

A new [TextBlob](#).

__init__(*tokenizer=None, pos_tagger=None, np_extractor=None, analyzer=None, parser=None, classifier=None*)

__repr__()

Return `repr(self)`.

__str__()

Return `str(self)`.

3.7.10 File Formats

File formats for training and testing data.

Includes a registry of valid file formats. New file formats can be added to the registry like so:

```
from textblob import formats

class PipeDelimitedFormat(formats.DelimitedFormat):
    delimiter = "|"

formats.register("psv", PipeDelimitedFormat)
```

Once a format has been registered, classifiers will be able to read data files with that format.

```
from textblob.classifiers import NaiveBayesAnalyzer

with open("training_data.psv", "r") as fp:
    cl = NaiveBayesAnalyzer(fp, format="psv")
```

class textblob.formats.BaseFormat(fp, **kwargs)

Interface for format classes. Individual formats can decide on the composition and meaning of ****kwargs**.

Parameters

fp (*File*) – A file-like object.

Changed in version 0.9.0: Constructor receives a file pointer rather than a file path.

classmethod detect(stream: str)

Detect the file format given a filename. Return True if a stream is this file format.

Changed in version 0.9.0: Changed from a static method to a class method.

to_iterable()

Return an iterable object from the data.

class textblob.formats.CSV(fp, **kwargs)

CSV format. Assumes each row is of the form **text,label**.

```
Today is a good day,pos
I hate this car.,pos
```

classmethod detect(stream)

Return True if stream is valid.

to_iterable()

Return an iterable object from the data.

class textblob.formats.DelimitedFormat(fp, **kwargs)

A general character-delimited format.

classmethod detect(stream)

Return True if stream is valid.

to_iterable()

Return an iterable object from the data.

class textblob.formats.JSON(fp, **kwargs)

JSON format.

Assumes that JSON is formatted as an array of objects with `text` and `label` properties.

```
[
    {"text": "Today is a good day.", "label": "pos"},
    {"text": "I hate this car.", "label": "neg"},
]
```

classmethod detect(stream: str | bytes | bytearray)

Return True if stream is valid JSON.

to_iterable()

Return an iterable object from the JSON data.

class textblob.formats.TSV(fp, **kwargs)

TSV format. Assumes each row is of the form `text label`.

classmethod detect(stream)

Return True if stream is valid.

to_iterable()

Return an iterable object from the data.

textblob.formats.detect(fp, max_read=1024)

Attempt to detect a file's format, trying each of the supported formats. Return the format class that was detected. If no format is detected, return `None`.

textblob.formats.get_registry()

Return a dictionary of registered formats.

textblob.formats.register(name, format_class)

Register a new format.

Parameters

- **name** (str) – The name that will be used to refer to the format, e.g. 'csv'
- **format_class** (type) – The format class to register.

3.7.11 Wordnet

Wordnet interface. Contains classes for creating Synsets and Lemmas directly.

Added in version 0.7.0.

textblob.wordnet.Lemma(name, lang='eng')

Lemma constructor

textblob.wordnet.Synset(name)

Synset constructor

textblob.wordnet.wordnet = <WordNetCorpusReader in
'/Users/karstenw/nltk_data/corpora/wordnet.zip/wordnet/'>

wordnet module from nltk

3.7.12 Exceptions

exception textblob.exceptions.TextBlobError

A TextBlob-related error.

exception textblob.exceptions.MissingCorpusError(*message="\nLooks like you are missing some required data for this feature.\n\nTo download the necessary data, simply run\n\n python -m textblob.download_corpora\n\nor use the NLTK downloader to download the missing data: http://nltk.org/data.html\n\nIf this doesn't fix the problem, file an issue at https://github.com/sloria/TextBlob/issues.\n", *args, **kwargs*)

Exception thrown when a user tries to use a feature that requires a dataset or model that the user does not have on their system.

exception textblob.exceptions.DeprecationError

Raised when user uses a deprecated feature.

exception textblob.exceptions.TranslatorError

Raised when an error occurs during language translation or detection.

exception textblob.exceptions.NotTranslated

Raised when text is unchanged after translation. This may be due to the language being unsupported by the translator.

exception textblob.exceptions.FormatError

Raised if a data file with an unsupported format is passed to a classifier.

PROJECT INFO

PYTHON MODULE INDEX

t

- `textblob.base`, [38](#)
- `textblob.blob`, [9](#)
- `textblob.classifiers`, [42](#)
- `textblob.en.np_extractors`, [41](#)
- `textblob.en.parsers`, [42](#)
- `textblob.en.sentiments`, [42](#)
- `textblob.en.taggers`, [40](#)
- `textblob.exceptions`, [54](#)
- `textblob.formats`, [52](#)
- `textblob.tokenizers`, [39](#)
- `textblob.wordnet`, [53](#)

Symbols

`__call__()` (*textblob.blob.Blobber method*), 51
`__init__()` (*textblob.blob.Blobber method*), 51
`__repr__()` (*textblob.blob.Blobber method*), 51
`__str__()` (*textblob.blob.Blobber method*), 51

A

`accuracy()` (*textblob.classifiers.DecisionTreeClassifier method*), 44
`accuracy()` (*textblob.classifiers.MaxEntClassifier method*), 45
`accuracy()` (*textblob.classifiers.NaiveBayesClassifier method*), 48
`accuracy()` (*textblob.classifiers.NLTKClassifier method*), 47
`accuracy()` (*textblob.classifiers.PositiveNaiveBayesClassifier method*), 50
`accuracy()` (*textblob.en.np_extractors.ChunkParser method*), 41
`analyze()` (*textblob.base.BaseSentimentAnalyzer method*), 38
`analyze()` (*textblob.en.sentiments.NaiveBayesAnalyzer method*), 42
`analyze()` (*textblob.en.sentiments.PatternAnalyzer method*), 42
`append()` (*textblob.blob.WordList method*), 36

B

`BaseBlob` (*class in textblob.blob*), 20
`BaseClassifier` (*class in textblob.classifiers*), 43
`BaseFormat` (*class in textblob.formats*), 52
`BaseNPExtractor` (*class in textblob.base*), 38
`BaseParser` (*class in textblob.base*), 38
`BaseSentimentAnalyzer` (*class in textblob.base*), 38
`BaseTagger` (*class in textblob.base*), 38
`BaseTokenizer` (*class in textblob.base*), 38
`basic_extractor()` (*in module textblob.classifiers*), 50
`Blobber` (*class in textblob.blob*), 23, 51

C

`capitalize()` (*textblob.blob.Word method*), 30
`casefold()` (*textblob.blob.Word method*), 30

`center()` (*textblob.blob.Word method*), 30
`ChunkParser` (*class in textblob.en.np_extractors*), 41
`classifier` (*textblob.classifiers.BaseClassifier attribute*), 43
`classifier` (*textblob.classifiers.DecisionTreeClassifier attribute*), 44
`classifier` (*textblob.classifiers.MaxEntClassifier attribute*), 45
`classifier` (*textblob.classifiers.NaiveBayesClassifier attribute*), 48
`classifier` (*textblob.classifiers.NLTKClassifier attribute*), 47
`classifier` (*textblob.classifiers.PositiveNaiveBayesClassifier attribute*), 50
`classify()` (*textblob.blob.BaseBlob method*), 20
`classify()` (*textblob.blob.Sentence method*), 24
`classify()` (*textblob.blob.TextBlob method*), 27
`classify()` (*textblob.classifiers.BaseClassifier method*), 43
`classify()` (*textblob.classifiers.DecisionTreeClassifier method*), 44
`classify()` (*textblob.classifiers.MaxEntClassifier method*), 45
`classify()` (*textblob.classifiers.NaiveBayesClassifier method*), 48
`classify()` (*textblob.classifiers.NLTKClassifier method*), 47
`classify()` (*textblob.classifiers.PositiveNaiveBayesClassifier method*), 50
`clear()` (*textblob.blob.WordList method*), 37
`ConllExtractor` (*class in textblob.en.np_extractors*), 41
`contains_extractor()` (*in module textblob.classifiers*), 51
`copy()` (*textblob.blob.WordList method*), 37
`correct()` (*textblob.blob.BaseBlob method*), 20
`correct()` (*textblob.blob.Sentence method*), 24
`correct()` (*textblob.blob.TextBlob method*), 27
`correct()` (*textblob.blob.Word method*), 31
`count()` (*textblob.blob.Word method*), 31
`count()` (*textblob.blob.WordList method*), 37
`CSV` (*class in textblob.formats*), 52

D

`DecisionTreeClassifier` (class in `textblob.classifiers`), 44
`define()` (`textblob.blob.Word` method), 31
`definitions` (`textblob.blob.Word` attribute), 31
`DelimitedFormat` (class in `textblob.formats`), 52
`DeprecationError`, 54
`detect()` (in module `textblob.formats`), 53
`detect()` (`textblob.formats.BaseFormat` class method), 52
`detect()` (`textblob.formats.CSV` class method), 52
`detect()` (`textblob.formats.DelimitedFormat` class method), 52
`detect()` (`textblob.formats.JSON` class method), 53
`detect()` (`textblob.formats.TSV` class method), 53
`dict` (`textblob.blob.Sentence` property), 24

E

`encode()` (`textblob.blob.Word` method), 31
`end` (`textblob.blob.Sentence` attribute), 24
`end_index` (`textblob.blob.Sentence` attribute), 24
`ends_with()` (`textblob.blob.BaseBlob` method), 20
`ends_with()` (`textblob.blob.Sentence` method), 24
`ends_with()` (`textblob.blob.TextBlob` method), 27
`endswith()` (`textblob.blob.BaseBlob` method), 20
`endswith()` (`textblob.blob.Sentence` method), 24
`endswith()` (`textblob.blob.TextBlob` method), 27
`endswith()` (`textblob.blob.Word` method), 31
`evaluate()` (`textblob.en.np_extractors.ChunkParser` method), 41
`expandtabs()` (`textblob.blob.Word` method), 31
`extend()` (`textblob.blob.WordList` method), 37
`extract()` (`textblob.base.BaseNPExtractor` method), 38
`extract()` (`textblob.en.np_extractors.ConllExtractor` method), 41
`extract()` (`textblob.en.np_extractors.FastNPExtractor` method), 41
`extract_features()` (`textblob.classifiers.BaseClassifier` method), 43
`extract_features()` (`textblob.classifiers.DecisionTreeClassifier` method), 44
`extract_features()` (`textblob.classifiers.MaxEntClassifier` method), 46
`extract_features()` (`textblob.classifiers.NaiveBayesClassifier` method), 48
`extract_features()` (`textblob.classifiers.NLTKClassifier` method), 47
`extract_features()` (`textblob.classifiers.PositiveNaiveBayesClassifier` method), 50

F

`FastNPExtractor` (class in `textblob.en.np_extractors`), 41

`find()` (`textblob.blob.BaseBlob` method), 20
`find()` (`textblob.blob.Sentence` method), 24
`find()` (`textblob.blob.TextBlob` method), 27
`find()` (`textblob.blob.Word` method), 31
`format()` (`textblob.blob.BaseBlob` method), 20
`format()` (`textblob.blob.Sentence` method), 24
`format()` (`textblob.blob.TextBlob` method), 27
`format()` (`textblob.blob.Word` method), 32
`format_map()` (`textblob.blob.Word` method), 32
`FormatError`, 54

G

`get_registry()` (in module `textblob.formats`), 53
`get_synsets()` (`textblob.blob.Word` method), 32
`grammar()` (`textblob.en.np_extractors.ChunkParser` method), 41

I

`index()` (`textblob.blob.BaseBlob` method), 20
`index()` (`textblob.blob.Sentence` method), 24
`index()` (`textblob.blob.TextBlob` method), 27
`index()` (`textblob.blob.Word` method), 32
`index()` (`textblob.blob.WordList` method), 37
`informative_features()` (`textblob.classifiers.NaiveBayesClassifier` method), 48
`insert()` (`textblob.blob.WordList` method), 37
`isalnum()` (`textblob.blob.Word` method), 32
`isalpha()` (`textblob.blob.Word` method), 32
`isascii()` (`textblob.blob.Word` method), 32
`isdecimal()` (`textblob.blob.Word` method), 32
`isdigit()` (`textblob.blob.Word` method), 32
`isidentifier()` (`textblob.blob.Word` method), 32
`islower()` (`textblob.blob.Word` method), 33
`isnumeric()` (`textblob.blob.Word` method), 33
`isprintable()` (`textblob.blob.Word` method), 33
`isspace()` (`textblob.blob.Word` method), 33
`istitle()` (`textblob.blob.Word` method), 33
`isupper()` (`textblob.blob.Word` method), 33
`itokenize()` (`textblob.base.BaseTokenizer` method), 38
`itokenize()` (`textblob.tokenizers.SentenceTokenizer` method), 39
`itokenize()` (`textblob.tokenizers.WordTokenizer` method), 39

J

`join()` (`textblob.blob.BaseBlob` method), 20
`join()` (`textblob.blob.Sentence` method), 24
`join()` (`textblob.blob.TextBlob` method), 28
`join()` (`textblob.blob.Word` method), 33
`JSON` (class in `textblob.formats`), 52
`json` (`textblob.blob.TextBlob` property), 28

L

[labels\(\)](#) (*textblob.classifiers.BaseClassifier* method), 43
[labels\(\)](#) (*textblob.classifiers.DecisionTreeClassifier* method), 44
[labels\(\)](#) (*textblob.classifiers.MaxEntClassifier* method), 46
[labels\(\)](#) (*textblob.classifiers.NaiveBayesClassifier* method), 48
[labels\(\)](#) (*textblob.classifiers.NLTKClassifier* method), 47
[labels\(\)](#) (*textblob.classifiers.PositiveNaiveBayesClassifier* method), 50
[lemma](#) (*textblob.blob.Word* attribute), 33
[Lemma\(\)](#) (in module *textblob.wordnet*), 53
[lemmatize\(\)](#) (*textblob.blob.Word* method), 33
[lemmatize\(\)](#) (*textblob.blob.WordList* method), 37
[ljust\(\)](#) (*textblob.blob.Word* method), 33
[lower\(\)](#) (*textblob.blob.BaseBlob* method), 21
[lower\(\)](#) (*textblob.blob.Sentence* method), 24
[lower\(\)](#) (*textblob.blob.TextBlob* method), 28
[lower\(\)](#) (*textblob.blob.Word* method), 33
[lower\(\)](#) (*textblob.blob.WordList* method), 37
[lstrip\(\)](#) (*textblob.blob.Word* method), 34

M

[maketrans\(\)](#) (*textblob.blob.Word* static method), 34
[MaxEntClassifier](#) (class in *textblob.classifiers*), 45
[MissingCorpusError](#), 54
[module](#)

- [textblob.base](#), 38
- [textblob.blob](#), 9, 19
- [textblob.classifiers](#), 42
- [textblob.en.np_extractors](#), 41
- [textblob.en.parsers](#), 42
- [textblob.en.sentiments](#), 42
- [textblob.en.taggers](#), 40
- [textblob.exceptions](#), 54
- [textblob.formats](#), 52
- [textblob.tokenizers](#), 39
- [textblob.wordnet](#), 53

N

[NaiveBayesAnalyzer](#) (class in *textblob.en.sentiments*), 42
[NaiveBayesClassifier](#) (class in *textblob.classifiers*), 47
[ngrams\(\)](#) (*textblob.blob.BaseBlob* method), 21
[ngrams\(\)](#) (*textblob.blob.Sentence* method), 24
[ngrams\(\)](#) (*textblob.blob.TextBlob* method), 28
[nltk_class](#) (*textblob.classifiers.DecisionTreeClassifier* attribute), 44
[nltk_class](#) (*textblob.classifiers.MaxEntClassifier* attribute), 46
[nltk_class](#) (*textblob.classifiers.NaiveBayesClassifier* attribute), 48
[nltk_class](#) (*textblob.classifiers.NLTKClassifier* attribute), 47
[nltk_class](#) (*textblob.classifiers.PositiveNaiveBayesClassifier* attribute), 50
[NLTKClassifier](#) (class in *textblob.classifiers*), 46
[NLTKTagger](#) (class in *textblob.en.taggers*), 40
[NotTranslated](#), 54
[noun_phrases](#) (*textblob.blob.BaseBlob* attribute), 21
[noun_phrases](#) (*textblob.blob.Sentence* attribute), 24
[noun_phrases](#) (*textblob.blob.TextBlob* attribute), 28
[np_counts](#) (*textblob.blob.BaseBlob* attribute), 21
[np_counts](#) (*textblob.blob.Sentence* attribute), 25
[np_counts](#) (*textblob.blob.TextBlob* attribute), 28

P

[parse\(\)](#) (*textblob.base.BaseParser* method), 38
[parse\(\)](#) (*textblob.blob.BaseBlob* method), 21
[parse\(\)](#) (*textblob.blob.Sentence* method), 25
[parse\(\)](#) (*textblob.blob.TextBlob* method), 28
[parse\(\)](#) (*textblob.en.np_extractors.ChunkParser* method), 41
[parse\(\)](#) (*textblob.en.parsers.PatternParser* method), 42
[parse_all\(\)](#) (*textblob.en.np_extractors.ChunkParser* method), 41
[parse_one\(\)](#) (*textblob.en.np_extractors.ChunkParser* method), 41
[parse_sents\(\)](#) (*textblob.en.np_extractors.ChunkParser* method), 41
[partition\(\)](#) (*textblob.blob.Word* method), 34
[PatternAnalyzer](#) (class in *textblob.en.sentiments*), 42
[PatternParser](#) (class in *textblob.en.parsers*), 42
[PatternTagger](#) (class in *textblob.en.taggers*), 40
[pluralize\(\)](#) (*textblob.blob.Word* method), 34
[pluralize\(\)](#) (*textblob.blob.WordList* method), 37
[polarity](#) (*textblob.blob.BaseBlob* attribute), 21
[polarity](#) (*textblob.blob.Sentence* attribute), 25
[polarity](#) (*textblob.blob.TextBlob* attribute), 28
[pop\(\)](#) (*textblob.blob.WordList* method), 37
[pos_tags](#) (*textblob.blob.BaseBlob* attribute), 21
[pos_tags](#) (*textblob.blob.Sentence* attribute), 25
[pos_tags](#) (*textblob.blob.TextBlob* attribute), 28
[PositiveNaiveBayesClassifier](#) (class in *textblob.classifiers*), 49
[pprint\(\)](#) (*textblob.classifiers.DecisionTreeClassifier* method), 44
[pretty_format\(\)](#) (*textblob.classifiers.DecisionTreeClassifier* method), 44
[prob_classify\(\)](#) (*textblob.classifiers.MaxEntClassifier* method), 46
[prob_classify\(\)](#) (*textblob.classifiers.NaiveBayesClassifier* method), 48

`pseudocode()` (*textblob.classifiers.DecisionTreeClassifier* method), 45

R

`raw_sentences` (*textblob.blob.TextBlob* property), 28

`register()` (in module *textblob.formats*), 53

`remove()` (*textblob.blob.WordList* method), 37

`removeprefix()` (*textblob.blob.Word* method), 34

`removesuffix()` (*textblob.blob.Word* method), 34

`replace()` (*textblob.blob.BaseBlob* method), 21

`replace()` (*textblob.blob.Sentence* method), 25

`replace()` (*textblob.blob.TextBlob* method), 29

`replace()` (*textblob.blob.Word* method), 34

`RETURN_TYPE` (*textblob.en.sentiments.NaiveBayesAnalyzer* attribute), 42

`RETURN_TYPE` (*textblob.en.sentiments.PatternAnalyzer* attribute), 42

`reverse()` (*textblob.blob.WordList* method), 37

`rfind()` (*textblob.blob.BaseBlob* method), 21

`rfind()` (*textblob.blob.Sentence* method), 25

`rfind()` (*textblob.blob.TextBlob* method), 29

`rfind()` (*textblob.blob.Word* method), 34

`rindex()` (*textblob.blob.BaseBlob* method), 21

`rindex()` (*textblob.blob.Sentence* method), 25

`rindex()` (*textblob.blob.TextBlob* method), 29

`rindex()` (*textblob.blob.Word* method), 34

`rjust()` (*textblob.blob.Word* method), 34

`rpartition()` (*textblob.blob.Word* method), 35

`rsplit()` (*textblob.blob.Word* method), 35

`rstrip()` (*textblob.blob.Word* method), 35

S

`sent_tokenize()` (in module *textblob.tokenizers*), 40

`Sentence` (class in *textblob.blob*), 23

`sentences` (*textblob.blob.TextBlob* attribute), 29

`SentenceTokenizer` (class in *textblob.tokenizers*), 39

`sentiment` (*textblob.blob.BaseBlob* attribute), 22

`sentiment` (*textblob.blob.Sentence* attribute), 25

`sentiment` (*textblob.blob.TextBlob* attribute), 29

`sentiment_assessments` (*textblob.blob.BaseBlob* attribute), 22

`sentiment_assessments` (*textblob.blob.Sentence* attribute), 25

`sentiment_assessments` (*textblob.blob.TextBlob* attribute), 29

`serialized` (*textblob.blob.TextBlob* property), 29

`show_informative_features()`
(*textblob.classifiers.NaiveBayesClassifier* method), 48

`singularize()` (*textblob.blob.Word* method), 35

`singularize()` (*textblob.blob.WordList* method), 37

`sort()` (*textblob.blob.WordList* method), 37

`span_tokenize()` (*textblob.tokenizers.SentenceTokenizer* method), 39

`span_tokenize()` (*textblob.tokenizers.WordTokenizer* method), 40

`span_tokenize_sents()`
(*textblob.tokenizers.SentenceTokenizer* method), 39

`span_tokenize_sents()`
(*textblob.tokenizers.WordTokenizer* method), 40

`spellcheck()` (*textblob.blob.Word* method), 35

`split()` (*textblob.blob.BaseBlob* method), 22

`split()` (*textblob.blob.Sentence* method), 26

`split()` (*textblob.blob.TextBlob* method), 29

`split()` (*textblob.blob.Word* method), 35

`splitlines()` (*textblob.blob.Word* method), 35

`start` (*textblob.blob.Sentence* attribute), 26

`start_index` (*textblob.blob.Sentence* attribute), 26

`starts_with()` (*textblob.blob.BaseBlob* method), 22

`starts_with()` (*textblob.blob.Sentence* method), 26

`starts_with()` (*textblob.blob.TextBlob* method), 29

`startswith()` (*textblob.blob.BaseBlob* method), 22

`startswith()` (*textblob.blob.Sentence* method), 26

`startswith()` (*textblob.blob.TextBlob* method), 29

`startswith()` (*textblob.blob.Word* method), 36

`stem()` (*textblob.blob.Word* method), 36

`stem()` (*textblob.blob.WordList* method), 38

`strip()` (*textblob.blob.BaseBlob* method), 22

`strip()` (*textblob.blob.Sentence* method), 26

`strip()` (*textblob.blob.TextBlob* method), 29

`strip()` (*textblob.blob.Word* method), 36

`subjectivity` (*textblob.blob.BaseBlob* attribute), 22

`subjectivity` (*textblob.blob.Sentence* attribute), 26

`subjectivity` (*textblob.blob.TextBlob* attribute), 29

`swapcase()` (*textblob.blob.Word* method), 36

`Synset()` (in module *textblob.wordnet*), 53

`synsets` (*textblob.blob.Word* attribute), 36

T

`tag()` (*textblob.base.BaseTagger* method), 38

`tag()` (*textblob.en.taggers.NLTKTagger* method), 40

`tag()` (*textblob.en.taggers.PatternTagger* method), 40

`tags` (*textblob.blob.BaseBlob* attribute), 22

`tags` (*textblob.blob.Sentence* attribute), 26

`tags` (*textblob.blob.TextBlob* attribute), 29

`TextBlob` (class in *textblob.blob*), 27

`textblob.base`
module, 38

`textblob.blob`
module, 9, 19

`textblob.classifiers`
module, 42

`textblob.en.np_extractors`
module, 41

`textblob.en.parsers`
module, 42

textblob.en.sentiments
 module, 42
 textblob.en.taggers
 module, 40
 textblob.exceptions
 module, 54
 textblob.formats
 module, 52
 textblob.tokenizers
 module, 39
 textblob.wordnet
 module, 53
 TextBlobError, 54
 title() (*textblob.blob.BaseBlob* method), 22
 title() (*textblob.blob.Sentence* method), 26
 title() (*textblob.blob.TextBlob* method), 30
 title() (*textblob.blob.Word* method), 36
 to_iterable() (*textblob.formats.BaseFormat* method), 52
 to_iterable() (*textblob.formats.CSV* method), 52
 to_iterable() (*textblob.formats.DelimitedFormat* method), 52
 to_iterable() (*textblob.formats.JSON* method), 53
 to_iterable() (*textblob.formats.TSV* method), 53
 to_json() (*textblob.blob.TextBlob* method), 30
 tokenize() (*textblob.base.BaseTokenizer* method), 38
 tokenize() (*textblob.blob.BaseBlob* method), 23
 tokenize() (*textblob.blob.Sentence* method), 26
 tokenize() (*textblob.blob.TextBlob* method), 30
 tokenize() (*textblob.tokenizers.SentenceTokenizer* method), 39
 tokenize() (*textblob.tokenizers.WordTokenizer* method), 40
 tokenize_sents() (*textblob.tokenizers.SentenceTokenizer* method), 39
 tokenize_sents() (*textblob.tokenizers.WordTokenizer* method), 40
 tokens (*textblob.blob.BaseBlob* attribute), 23
 tokens (*textblob.blob.Sentence* attribute), 26
 tokens (*textblob.blob.TextBlob* attribute), 30
 train() (*textblob.classifiers.BaseClassifier* method), 44
 train() (*textblob.classifiers.DecisionTreeClassifier* method), 45
 train() (*textblob.classifiers.MaxEntClassifier* method), 46
 train() (*textblob.classifiers.NaiveBayesClassifier* method), 49
 train() (*textblob.classifiers.NLTKClassifier* method), 47
 train() (*textblob.classifiers.PositiveNaiveBayesClassifier* method), 50
 train() (*textblob.en.np_extractors.ChunkParser* method), 41
 train() (*textblob.en.sentiments.NaiveBayesAnalyzer* method), 42
 translate() (*textblob.blob.Word* method), 36
 TranslatorError, 54
 TSV (*class in textblob.formats*), 53

U

 update() (*textblob.classifiers.DecisionTreeClassifier* method), 45
 update() (*textblob.classifiers.MaxEntClassifier* method), 46
 update() (*textblob.classifiers.NaiveBayesClassifier* method), 49
 update() (*textblob.classifiers.NLTKClassifier* method), 47
 update() (*textblob.classifiers.PositiveNaiveBayesClassifier* method), 50
 upper() (*textblob.blob.BaseBlob* method), 23
 upper() (*textblob.blob.Sentence* method), 27
 upper() (*textblob.blob.TextBlob* method), 30
 upper() (*textblob.blob.Word* method), 36
 upper() (*textblob.blob.WordList* method), 38

W

 Word (*class in textblob.blob*), 30
 word_counts (*textblob.blob.BaseBlob* attribute), 23
 word_counts (*textblob.blob.Sentence* attribute), 27
 word_counts (*textblob.blob.TextBlob* attribute), 30
 word_tokenize() (*in module textblob.tokenizers*), 40
 WordList (*class in textblob.blob*), 36
 wordnet (*in module textblob.wordnet*), 53
 words (*textblob.blob.BaseBlob* attribute), 23
 words (*textblob.blob.Sentence* attribute), 27
 words (*textblob.blob.TextBlob* attribute), 30
 WordTokenizer (*class in textblob.tokenizers*), 39

Z

 zfill() (*textblob.blob.Word* method), 36